

INFORME DE EVIDENCIAS Y CONTROL DE VERSIONES

Integrantes:

RONALD CRUZ CHINO

Denilson Davis Machaca Flores

Daniel Wilfredo Pomarez Huaranca

Este documento recopila las evidencias técnicas y capturas de pantalla correspondientes al desarrollo del proyecto de programación en Python, estructurado mediante control de versiones en Git, refactorización de código con manejo de excepciones y pruebas exhaustivas de validación.

1. Control de Versiones (Historial de Commits)

Se muestra el registro cronológico del repositorio utilizando Git (`git log --oneline`), reflejando la evolución incremental del trabajo desde el código base hasta la robustez final.

```
print("=== REGISTRO DE USUARIO ===")

nombre = input("Ingrese su nombre: ")
edad = int(input("Ingrese su edad: "))
nota = float(input("Ingrese su nota: "))
opcion = int(input("Ingrese una opción (1-3)"))

if edad >= 18:
    (function) def print(
else
    *values: object,
    sep: str | None = " ",
    end: str | None = "\n",
    file: SupportsWrite[str] | None =
    flush: Literal[False] = False
    if o ) -> None
elif Prints the values to a stream, or to sys.stdout by
    sep
elif string inserted between values, default a spa
else end
    print("Opción no válida")

print("Proceso finalizado.")
```

```
PS C:\Users\rc602\OneDrive\Desktop\trabajos> git log --oneline
7350f3a (HEAD -> master) Commit 5: incorporación de documentación y validaciones
cc851ca Commit 3: incorporación de manejo de errores
12ab8ea Commit 2: incorporación de validaciones
e2f6b5d Commit 1: código original
PS C:\Users\rc602\OneDrive\Desktop\trabajos> git status
```

2. Código Original / Ejecución Base

Ejecución preliminar del script en la terminal de PowerShell, evidenciando el funcionamiento inicial previo a la incorporación de filtros estrictos y validaciones de datos.

```
def pedir_real(mensaje, minimo, maximo):  
    while True:  
        try:  
            valor = float(input(mensaje))  
  
            if minimo <= valor <= maximo:  
                return valor  
  
            print(f"Error: ingrese un valor entre {minimo} y {maximo}")  
  
        except ValueError:  
            print("Error: debe ingresar un número válido. Intente de nuevo.")
```

3. Código Mejorado (Manejo de Errores y Validaciones)

Fragmento de código correspondiente a la función auxiliar refactorizada (pedir_real), la cual implementa bloques try-except y control de rangos para evitar fallos por entradas inválidas.

```

* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\rc602\OneDrive\Desktop\trabajos> python c
=== REGISTRO DE USUARIO ===
Ingrese su nombre: johan
Ingrese su edad: 12
Ingrese su nota: 45
Ingrese una opción (1-3): 2
johan es menor de edad.
Nota registrada: 45.0
Modificar información
Proceso finalizado.
PS C:\Users\rc602\OneDrive\Desktop\trabajos>

```

Figura: Implementación de función con manejo de excepciones

4. Pruebas de Validación y Ejecución Correcta

Demostración final del sistema seguro en funcionamiento. Se valida el ingreso correcto de parámetros (nombre, edad y notas con restricciones numéricas) y la presentación estructurada de la información registrada.

```

=== REGISTRO DE USUARIO SEGURO ===
Ingrese su nombre: johan
Ingrese su edad (0-120): 23
Ingrese su nota (0-100): 35
Ingrese una opción (1: Mostrar, 2: Modificar, 3: E
johan es mayor de edad.
Nota registrada: 35.00

--- INFORMACIÓN DEL USUARIO ---
Nombre: johan
Edad: 23
Nota: 35.00

Proceso finalizado correctamente.
PS C:\Users\rc602\OneDrive\Desktop\trabajos>

```

Figura: Pruebas de validación y resultados en consola